

Name : Ankita Pal

Reg. No: RA1711003010611

Subject : Compiler Design

Subject code : 15CS314J

Faculty : G.K. Sandhia

Assignment : Storage Allocation Strategies (Assignment-2)

Date : 08-04-2020

STORAGE ALLOCATION STRATEGIES

The different storage allocation strategies are:

1. Static Allocation - lays out storage for all data objects at compile time.
2. Stack allocation - manages the run-time storage as a stack
3. Heap allocation - allocates and deallocates storage as needed at run time from a data area known as heap

(I) Static Allocation :-

In static allocation, names are bound to storage as the program is compiled, so there is no need for a run-time support package. Since the bindings and bindings do not change at run-time, everytime a procedure is activated, its frames are bound to the same storage locations. Therefore values of local names are retained across activations of a procedure.

That is when control returns to a procedure the values of the locals are the same as they were when control left the last time. From the type of a name,

the compiler decides the amount of storage for the name and decides where the activation records go. At compile time, we can fill in the addresses at which the target code can find the data it operates on.

(II) Stack allocation of space :-

all compilers for languages that use procedures, functions or methods as units of user-defined actions manage at least part of their run-time memory as a stack. Each time a procedure is called, space for its local variables is pushed onto a stack, and when the procedure terminates, that is popped off the stack.

calling sequences:

Procedures called are implemented in what is called a calling sequence, which consists of code that allocates an activation record on the stack and enters information into its fields. A return sequence is similar to code to restore the state of machine so the calling procedure can continue its execution after the call. The code in calling sequence is often divided between the calling procedure (caller) and the procedure it calls (callee).

When designing calling sequences and the layout of activation records, the following principles are helpful:

Values communicated between caller and callee are generally placed at the beginning of the callee's activation record, so they are as close as possible to the caller's activation record.

- i) Fixed length items are generally placed in the middle. such is the control link, the access link, and the machine status fields.
- ii) Items whose size may not be known early enough are placed at the end of the activation record. The most common example is dynamically sized array, where the value of the callee's parameters determines the length of the array.
- iii) We must locate the top-of-stack pointer judiciously. A common approach is to have it point to the end of fixed-length fields in the activation record. Fixed-length data can be then accessed by fixed offsets, known to the intermediate-code generator, relative to the top-of-stack pointer.

The calling sequence and its division between caller and callee are as follows:

1. The caller evaluates the actual parameters.
2. The caller stores a return address and the old value of top-sp into the callee's activation record. The callee then increments the top-sp to the respective positions.
3. The callee saves the register values and other status information.
4. The callee initializes its local data and begins execution.

variable length data on stack :

The run-time memory management system must deal frequently with the allocation of space for objects, the sizes of which are not known at the compile time, but which are not known at the compile time, they are local to a procedure and thus may be allocated on the stack. The reason to prefer placing objects on the stack is that we avoid the expense of garbage collecting their space. The same scheme works for objects of any type if they are local to the procedure called and have a size that depends on the parameters of the call.

(III) Heap Allocation:-

- Stack Allocation strategy cannot be used if either is possible
1. The values of local names must be retained when an activation ends.
 2. A called activation outlives the caller.

Heap allocation parcels out pieces of contiguous storage, as needed for activation records or other objects. Pieces may be deallocated in any order, so over the time the heap will consist of alternate areas that are free and in use.

The record for an activation of procedure q is retained when the activation ends.

Therefore, the record for the new activation $q(1, 9)$ cannot follow that for s physically.

If the retained activation record for x is deallocated, there will be free space in the heap between the activation records for s and q .

Position in the activation tree	Activation records in the heap	Remarks
<pre> graph TD s --- x s --- q["q(1, q)"] </pre>		Retained activation record for x

Records for live activations need not be adjacent in heap.